

Sales Module — Business Logic Summary

Overview

All three sales documents — **Sales Order**, **Sales Bill (Invoice)**, and **Sales Return** — live on a single database table: `loagma_new.orders` . There is no separate invoices table or returns table. Line items live in `loagma_new.orders_item` .

1. Sales Order

What it is

A customer's request to purchase products. Created by admin/telecaller.

Lifecycle (order_state)

pending → registered → dispatched → delivered → billed
↘ cancelled (from any state)

Key fields (orders table)

Field	Purpose
order_id	Primary key
buyer_userid	Customer ID
buyer_name	Customer name
order_state	Current status
order_total	Total amount
bill_number	Bill/invoice number (set when billed)

Key fields (orders_item table)

Field	Purpose
item_id	Primary key
order_id	Parent order
product_id	Product
quantity	Original ordered qty
item_price	Unit price
qty_delivered	Qty actually delivered to customer (set at billing)
qty_returned	Cumulative qty returned by customer (incremented per return)

Qty computed values

left_qty = quantity - qty_delivered → still undelivered
available_to_return = qty_delivered - qty_returned → customer can still return this much

APIs

Method	Endpoint	Action
GET	/api/sales-orders	List orders (paginated, filterable)
POST	/api/sales-orders	Create new order
GET	/api/sales-orders/{id}	Get order with items
PUT	/api/sales-orders/{id}	Update order / record billing
DELETE	/api/sales-orders/{id}	Delete order

2. Sales Bill (Invoice)

What it is

Not a separate document — billing is a **state change on the Sales Order** (`order_state = 'billed'`). When the admin marks an order as billed, extra bill fields are recorded on the same `orders` row and `qty_delivered` is set per item.

What happens at billing

1. Admin opens the Sales Order form
2. Sets `order_state = 'billed'`
3. Fills in bill-specific fields (bill date, vehicle, narration, etc.)
4. Enters `qty_delivered` per item (how many units actually reached the customer)
5. Saves → `PUT /api/sales-orders/{id}` updates the order row

Bill fields on orders table

Field	Purpose
<code>bill_number</code>	Bill/invoice number
<code>Bill_Dt</code>	Bill date
<code>Department</code>	Department
<code>Bill_Narration</code>	Narration / remarks
<code>Bill_Vehicle</code>	Delivery vehicle
<code>Bill_Statement</code>	Statement reference
<code>bill_roff</code>	Round-off amount
<code>Doc_Year</code>	Financial year

UI behaviour

- The Flutter form shows a **"Qty Delivered" field per item** only when `order_state == 'billed'` (controlled by `isBillMode` getter in `SalesOrderFormController`)
- All bill fields are in the same Sales Order form, shown in a separate "Bill Details" section

3. Sales Return

What it is

A record that the customer sent back some or all of the delivered goods. Like billing, it is **not a separate document** — return data is stored as columns on the same `orders` row, and `qty_returned` is updated per item in `orders_item`.

What happens at a return

1. Admin opens the Sales Return form, picks the source order
2. Fills in return date, reason, and returned qty per item
3. Saves → `POST /api/sales-returns` writes return columns onto the order row and increments `qty_returned` on each returned item

Return fields on orders table

Field	Purpose
<code>Sales_Return_VoucherNo</code>	Return voucher number (e.g. SR/25-26/001)
<code>Sales_Return_Dt</code>	Return date
<code>Sales_Return_Reason</code>	Reason for return

Return field on orders_item table

Field	Purpose
<code>qty_returned</code>	Cumulative qty returned for this line item

Constraints

- One return record per order (since columns are on the order row)
- Editing a return reverses the old `qty_returned` values and re-applies the new ones
- Deleting a return clears the return columns and resets `qty_returned = 0` on all items

APIs

Method	Endpoint	Action
GET	/api/sales-returns/series	Get next voucher number
GET	/api/sales-returns	List orders that have a return (Sales_Return_VoucherNo IS NOT NULL)
POST	/api/sales-returns	Record a return against a source order
GET	/api/sales-returns/{order_id}	Get return details for an order
PUT	/api/sales-returns/{order_id}	Update return
DELETE	/api/sales-returns/{order_id}	Delete return, reset qty_returned

Full Qty Flow Example

1. Order created
quantity=10, qty_delivered=0, qty_returned=0
→ left_qty=10, available_to_return=0
2. Driver delivers 8 units (2 short)
Admin marks order BILLED, sets qty_delivered=8 per item
quantity=10, qty_delivered=8, qty_returned=0
→ left_qty=2, available_to_return=8
3. Customer returns 3 units
POST /api/sales-returns → qty_returned += 3
quantity=10, qty_delivered=8, qty_returned=3
→ left_qty=2, available_to_return=5
4. Customer returns 2 more units
PUT /api/sales-returns/{order_id} → reverses old, applies new
quantity=10, qty_delivered=8, qty_returned=5
→ left_qty=2, available_to_return=3

Database Tables Involved

Table	Manages
loagma_new.orders	Order header, bill fields, return fields
loagma_new.orders_item	Line items with qty tracking

No separate sales_invoices , sales_returns , or sales_return_items tables exist.

Flutter Controllers & Screens

Controller	Screen	Purpose
SalesOrderFormController	SalesOrderFormScreen	Create/edit order; billing mode
SalesReturnFormController	SalesReturnFormScreen	Record return against order
SalesReturnListController	SalesReturnListScreen	List all returns

SalesOrderFormController.isBillMode → true when status == 'BILLED' ; drives bill-specific UI fields including per-item Qty Delivered.